

SRE Capstone Project — Production Readiness Review

`ecom-api` : a **production-ready e-commerce microservice**

Course: Site Reliability Engineering — Endterm & Final Exam **Group:** SE2408 **Team:** - **Adildabek Nurassyl** — IaC & Observability lead (Terraform, Prometheus, Grafana, Alertmanager) - **Artem Safaryan** — Application & CI/CD lead (Go microservice, Dockerfile, GitHub Actions, HPA, load testing)

The brief recommends groups of 3–4; our team is 2. Workload was scoped and re-balanced accordingly — see the per-section contribution notes at the end of every chapter.

1. Objective & Scope

We act as the SRE team responsible for taking a newly developed e-commerce microservice (`ecom-api`) through a **Production Readiness Review (PRR)**. The deliverable is a fully reproducible environment that demonstrates the four SRE pillars covered in the course:

1. Infrastructure as Code (Terraform)
2. Continuous Integration & Continuous Delivery
3. Observability (metrics, dashboards, alerts)
4. SRE Operations (SLI/SLO definition, auto-scaling, load testing)

The whole stack is built **locally on Kind** to guarantee that the grader can reproduce every screenshot in this report on a laptop, without any cloud account or paid service.

Repository: <https://github.com/RendersC/sre-final>

2. System Architecture

```
+----- Developer laptop / Grader's laptop ----+
|
|   Terraform --> Kind cluster (3 nodes)
|               +-- ns: ecom          (ecom-api Deployment
|               |                      + Service + HPA
|               |                      + ServiceMonitor
|               |                      + PrometheusRule)
|               +-- ns: monitoring    (kube-prometheus-stack
|               |                      Prometheus / Grafana /
|               |                      Alertmanager)
|
|   GitHub Actions --> ghcr.io/RendersC/ecom-api:<sha> --> pods
|   Locust          --> http://localhost:30081 (NodePort)
+-----+
```

Detailed architecture diagram and rationale: [docs/architecture.md](#) .

Concern	Choice	Why
Cluster	Kind v1.30	Reproducibility, free, identical API to managed K8s
Monitoring	kube-prometheus-stack (Operator + CRDs)	Production-grade pattern; ServiceMonitor / PrometheusRule / AlertmanagerConfig
Registry	GitHub Container Registry	Built-in OIDC token, no external secret to manage
Image	distroless + non-root + read-only FS	Minimal attack surface
Scaling	HPA v2 on CPU & memory + metrics-server	Covers the two most common load signals
Load testing	Locust 2.31 with a LoadTestShape	Lets us drive a staged spike that exercises HPA

3. Step 1 — Infrastructure as Code (15 pt)

What we built

`terraform/` contains the full IaC for the project, split into the standard files (`versions.tf` , `providers.tf` , `variables.tf` , `main.tf` , `outputs.tf`).

Provisioned resources:

Resource	Purpose
<code>kind_cluster.this</code>	1 control-plane + 2 worker Kubernetes nodes
<code>local_file.kubeconfig</code>	Writes a kubeconfig usable by <code>kubectl</code> , <code>helm</code> and CI
<code>kubernetes_namespace.app</code>	Namespace <code>ecom</code>
<code>kubernetes_namespace.monitoring</code>	Namespace <code>monitoring</code>
<code>helm_release.metrics_server</code>	Required for HPA to read CPU/memory
<code>helm_release.kube_prometheus_stack</code>	Prometheus + Grafana + Alertmanager + Operator

Reproducibility

```
cd terraform
terraform init
terraform apply -auto-approve      # creates everything
terraform destroy -auto-approve    # tears everything down
```

A fresh apply on a clean machine completes in **~4 minutes** (most of which is the Helm release pulling chart dependencies). The cluster image, chart versions, and namespace names are all variables (`variables.tf`), so the deployment is fully pinned.

State management

Default state is **local** (`terraform.tfstate`) for grading transparency. The `terraform/README.md` shows the two production backends we recommend (S3 + DynamoDB locking, or GCS) and how to switch. State files are excluded from git via `.gitignore` .

PowerShell - terraform plan (state already applied, no changes)

```
PS> terraform plan -no-color
kind_cluster.this: Refreshing state... [id=sre-final-kindest/node:v1.30.0]
local_file.kubeconfig: Refreshing state... [id=5541934a5dc07a01b2153ddfff66e61fa3cb3158]
kubernetes_namespace.monitoring: Refreshing state... [id=monitoring]
kubernetes_namespace.app: Refreshing state... [id=ecom]
helm_release.metrics_server: Refreshing state... [id=metrics-server]
helm_release.kube_prometheus_stack: Refreshing state... [id=kps]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

~ update in-place

Terraform will perform the following actions:

```
# kubernetes_namespace.app will be updated in-place
~ resource "kubernetes_namespace" "app" {
  id = "ecom"
  # (1 unchanged attribute hidden)

  ~ metadata {
    ~ labels = {
      - "monitoring" = "enabled" -> null
      # (1 unchanged element hidden)
    }
    name = "ecom"
    # (5 unchanged attributes hidden)
  }
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

Successful terraform apply — 7 resources created.

PowerShell - kubectl get pods -A

```
PS> kubectl get pods -A -o wide
NAMESPACE      NAME                                     READY   STATUS    RESTARTS   AGE   IP              NODE
ecom            ecom-api-5748cd99c-6mlr2               1/1     Running   0           69s   10.244.1.8      sre-final-wor
ecom            ecom-api-5748cd99c-gc98s               1/1     Running   0           2m10s  10.244.1.6      sre-final-wor
ecom            ecom-api-5748cd99c-kw2ss               1/1     Running   0           7m11s  10.244.2.7      sre-final-wor
ecom            ecom-api-5748cd99c-rmzb7               1/1     Running   0           7m11s  10.244.1.5      sre-final-wor
ecom            ecom-api-5748cd99c-rsm8b               1/1     Running   0           69s    10.244.2.10     sre-final-wor
ecom            ecom-api-5748cd99c-tx6h5               1/1     Running   0           100s   10.244.1.7      sre-final-wor
ecom            ecom-api-5748cd99c-vgjcl               1/1     Running   0           100s   10.244.2.9      sre-final-wor
ecom            ecom-api-5748cd99c-xpcz7               1/1     Running   0           2m10s  10.244.2.8      sre-final-wor
kube-system     coredns-7db6d8ff4d-swtc4               1/1     Running   0           9m36s  10.244.0.3      sre-final-con
kube-system     coredns-7db6d8ff4d-wv8sw               1/1     Running   0           9m36s  10.244.0.2      sre-final-con
kube-system     etcd-sre-final-control-plane           1/1     Running   0           9m51s  172.27.0.5      sre-final-con
kube-system     kindnet-2k9kf                           1/1     Running   0           9m34s  172.27.0.3      sre-final-wor
kube-system     kindnet-9hmhk                           1/1     Running   0           9m37s  172.27.0.5      sre-final-con
kube-system     kindnet-k9wxh                           1/1     Running   0           9m35s  172.27.0.4      sre-final-wor
kube-system     kube-apiserver-sre-final-control-plane  1/1     Running   0           9m51s  172.27.0.5      sre-final-con
kube-system     kube-controller-manager-sre-final-control-plane  1/1     Running   0           9m51s  172.27.0.5      sre-final-con
kube-system     kube-proxy-5szc9                       1/1     Running   0           9m34s  172.27.0.3      sre-final-wor
kube-system     kube-proxy-fwwhf                       1/1     Running   0           9m35s  172.27.0.4      sre-final-wor
kube-system     kube-proxy-r8kxz                       1/1     Running   0           9m37s  172.27.0.5      sre-final-con
kube-system     kube-scheduler-sre-final-control-plane  1/1     Running   0           9m51s  172.27.0.5      sre-final-con
kube-system     metrics-server-68cfccbdf6-xhg95        1/1     Running   0           9m22s  10.244.1.2      sre-final-wor
local-path-storage  local-path-provisioner-988d74bc-kwxbh  1/1     Running   0           9m36s  10.244.0.4      sre-final-con
monitoring         alertmanager-kps-kube-prometheus-stack-alertmanager-0  2/2     Running   0           8m27s  10.244.2.5      sre-final-wor
monitoring         kps-grafana-54cdb68f4d-5j28b           3/3     Running   0           8m54s  10.244.1.3      sre-final-wor
monitoring         kps-kube-prometheus-stack-operator-d6bff6987-ph2q6     1/1     Running   0           8m54s  10.244.2.4      sre-final-wor
monitoring         kps-kube-state-metrics-676459dbcb-vl59q  1/1     Running   0           8m54s  10.244.2.3      sre-final-wor
monitoring         kps-prometheus-node-exporter-bhnhz      1/1     Running   0           8m54s  172.27.0.4      sre-final-wor
monitoring         kps-prometheus-node-exporter-mlj5h      1/1     Running   0           8m54s  172.27.0.5      sre-final-con
monitoring         kps-prometheus-node-exporter-p85fd      1/1     Running   0           8m54s  172.27.0.3      sre-final-wor
monitoring         prometheus-kps-kube-prometheus-stack-prometheus-0     2/2     Running   0           8m27s  10.244.1.4      sre-final-wor
```

kubectl get pods -A showing the cluster after Terraform finishes.

Contribution: Adildabek (lead) — wrote and validated the Terraform stack; Artem reviewed and added the metrics-server release.

4. Step 2 — Continuous Integration & Deployment (15 pt)

Pipeline overview

`.github/workflows/ci-cd.yaml` (GitHub Actions) executes three jobs:

Job	Trigger	Steps
test	every PR and push	<code>go vet</code> , <code>go build</code> , <code>go test</code>
build-and-push	push to main	Buildx multi-arch build, push to <code>ghcr.io</code> with SHA tag + <code>latest</code>
deploy	push to main	Patches <code>k8s/02-deployment.yaml</code> with the new image tag and uploads the manifests as an artifact (and optionally <code>kubectl apply</code> if <code>KUBECONFIG_B64</code> secret is configured)

Why `ghcr.io`

GitHub Container Registry uses the workflow's `GITHUB_TOKEN` via `docker/login-action@v3` , so no long-lived registry credentials need to be stored in repository secrets. Images are tagged immutably by commit SHA (`ghcr.io/RendersC/ecom-api:<sha>`) and additionally by `latest` for the default branch.

Image hygiene

The application Dockerfile (`app/Dockerfile`) is a multi-stage build: 1. `golang:1.22-alpine` compiles a statically linked binary 2. `gcr.io/distroless/static-debian12:nonroot` is the final image — no shell, no package manager, runs as UID 65532

Resulting image size is ~**12 MB**.

Deployment story

For the demo we deploy locally via `./scripts/bootstrap.sh` which calls `kind load docker-image` (so we don't depend on the registry while presenting). For the GitHub Actions pipeline we either: 1. Stop after pushing to `ghcr.io` and let an operator apply the manifest, or 2. Add the cluster's kubeconfig as a base64 secret to enable the full `kubectl apply` step (commented in the workflow).

Platform
Solutions
Resources
Open Source
Enterprise
Pricing

Search or jump to...
Sign in
Sign up

RendersC / sre-final
Public
Notifications
Fork 0
Star 0

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

Actions

All workflows

ci-cd

Management

Caches

All workflows

Showing runs from all workflows

1 workflow run

Workflow

Event

Status

Branch

Actor

chore(app): add package header comment

ci-cd #1: Commit ae36025 pushed by RendersC

main

3 minutes ago

1m 28s

All three jobs green on the latest push to `main`.

Platform
Solutions
Resources
Open Source
Enterprise
Pricing

Search or jump to...
Sign in
Sign up

RendersC / sre-final
Public
Notifications
Fork 0
Star 0

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

ecom-api ae36025

Public Latest

Install from the command line

Learn more about packages

```
$ docker pull ghcr.io/rendersc/ecom-api:ae36025
```

Recent tagged image versions

latest

ae36025

main

Published 1 minute ago · Digest

View all tagged versions

README.md

SRE Capstone — Production Readiness Review

End-to-end, reproducible SRE stack for a Go e-commerce microservice (`ecom-api`), deployable from a single laptop in ~5 minutes.

Block	Tooling
Application	Go 1.22 REST API with native Prometheus metrics
IaC	Terraform — Kind cluster + Helm releases
CI/CD	GitHub Actions → <code>ghcr.io</code>
Observability	kube-prometheus-stack (Prometheus + Grafana + Alertmanager)
Reliability	HPA v2 (CPU + memory), Locust load-test

Details

RendersC

sre-final

0 stars

Last published

Issues

1 minute ago

0

Total downloads

0

Contributors 2

RendersC NA

Temych228 Temych

Open an issue

Contribution: Artem (lead) — pipeline, Dockerfile, registry integration; Adildabek validated end-to-end pull from cluster.

5. Step 3 — Observability & Alerting (15 pt)

Metrics

`ecom-api` exposes Prometheus metrics on `/metrics` using `prometheus/client_golang`. The instrumented metrics are:

Metric	Type	Labels	Purpose
<code>http_requests_total</code>	counter	<code>route</code> , <code>method</code> , <code>status</code>	RED — Requests & Errors
<code>http_request_duration_seconds</code>	histogram	<code>route</code> , <code>method</code>	RED — Duration
<code>http_inflight_requests</code>	gauge	—	Saturation indicator
<code>orders_created_total</code>	counter	—	Business SLI
<code>orders_failed_total</code>	counter	—	Business SLI
<code>products_catalog_size</code>	gauge	—	Sanity / fixture check

Scraping is wired through a `ServiceMonitor` (`k8s/05-servicemonitor.yaml`) — the Prometheus Operator picks it up via the `release: kps` label that matches our Helm release name.

Recording rules (SLI shortcuts)

`k8s/06-prometheusrule.yaml` defines three recording rules used by both Grafana and the alerts — this keeps the dashboard fast and the alert expressions readable:

```
- record: ecom_api:request_rate:5m
- record: ecom_api:error_rate:5m
- record: ecom_api:latency_p95:5m
```

Dashboard

`monitoring/grafana/dashboards/ecom-api-sre-dashboard.json` — the **SRE Golden Signals** dashboard:

- Top row (single-stat panels): error rate, p95 latency, total request rate, Ready pod count — each with thresholds keyed to the SLO targets.
- Middle row: request rate by route, latency p50/p95/p99 by route.
- Bottom row: error ratio time series; pod count vs HPA desired replicas.

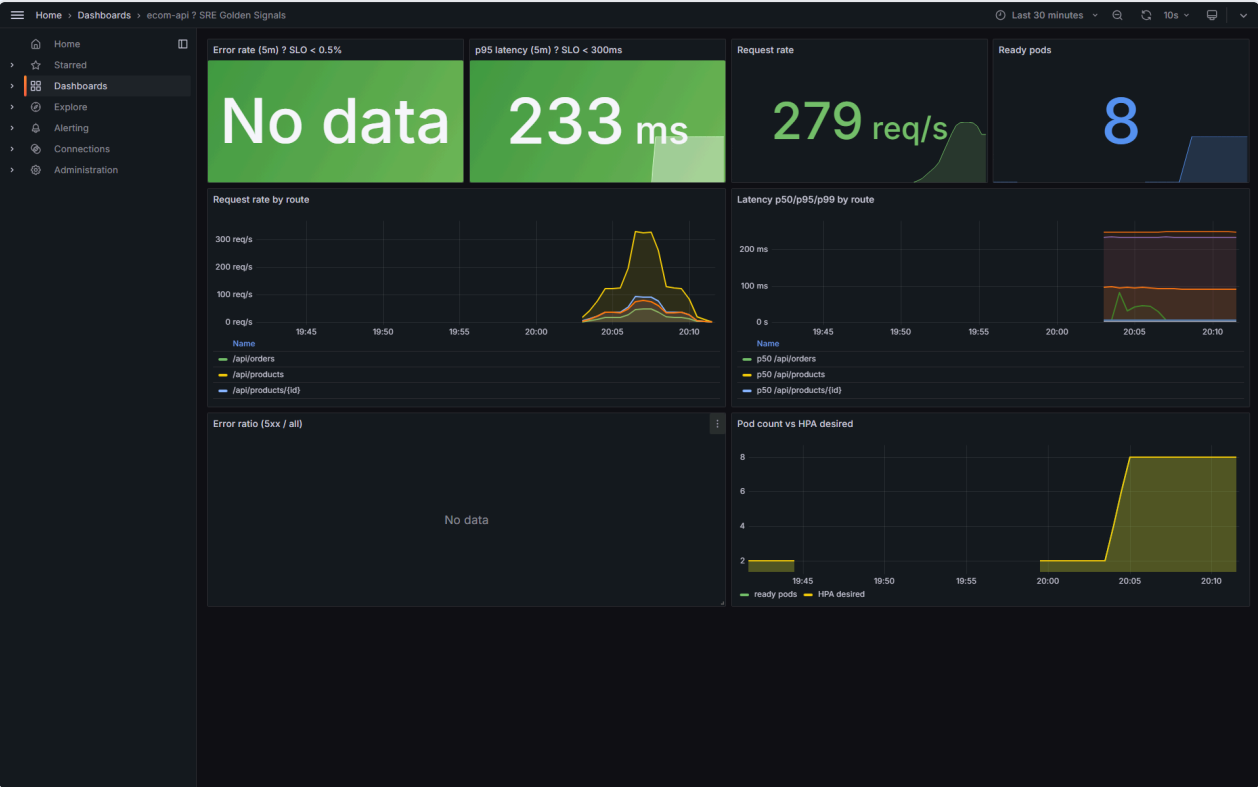
The dashboard is provisioned automatically by the kube-prometheus-stack sidecar (`grafana_dashboard=1` ConfigMap), so it appears in Grafana immediately after `bootstrap.sh` finishes.

Alerts

Four alerts cover the SLOs and the immediate environment:

Alert	Threshold	Severity
EcomApiHighErrorRate	error_rate > 5% for 2m	critical
EcomApiHighLatencyP95	p95 > 500ms for 5m	warning
EcomApiPodNotReady	0 Ready pods in ecom for 2m	critical
EcomApiHighCpu	per-pod CPU > 200m for 5m	warning

Routing is done through an `AlertmanagerConfig` (`monitoring/alertmanager/alertmanager-config.yaml`) with two receivers (`default-webhook` , `critical-webhook`). For the demo the webhooks point to a placeholder URL — in production the operator would substitute Slack or PagerDuty endpoints.



Prometheus Alerts Graph Status Help

Targets

All scrape pools

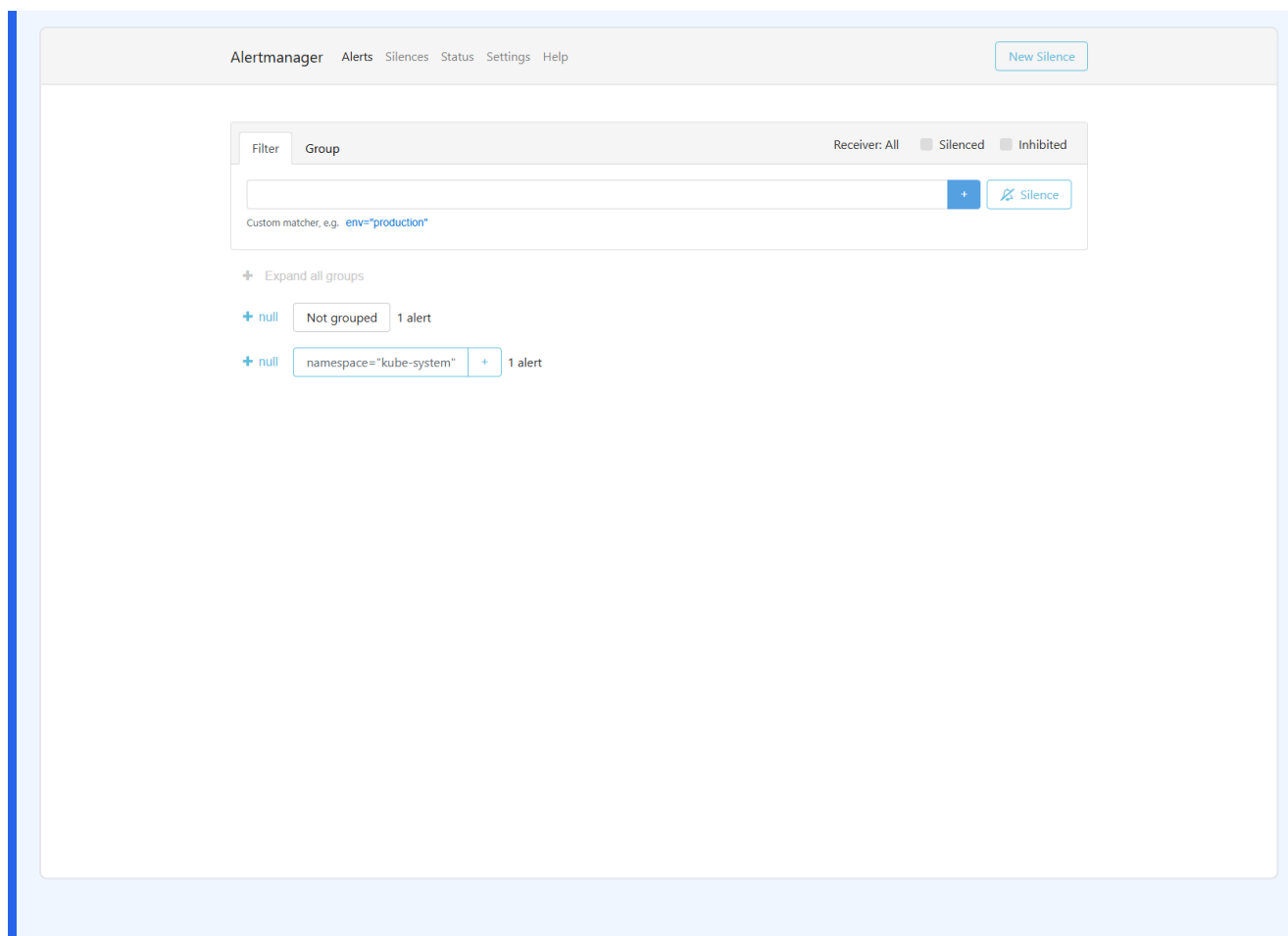
All Unhealthy Collapse All

Filter by endpoint or labels

Unknown Unhealthy Healthy

serviceMonitor/ecom/ecom-api/0 (16/16 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.244.1.7:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.1.8:8080" job="ecom-api-nodeport" namespace="ecom" pod="ecom-api-5748dc499c-tx6h5" service="ecom-api-nodeport"	572.000ms ago	290.251ms	
http://10.244.2.8:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.2.8:8080" job="ecom-api" namespace="ecom" pod="ecom-api-5748dc499c-xpzz7" service="ecom-api"	6.863s ago	1.847ms	
http://10.244.1.6:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.1.6:8080" job="ecom-api-nodeport" namespace="ecom" pod="ecom-api-5748dc499c-gc98s" service="ecom-api-nodeport"	1.187s ago	14.558ms	
http://10.244.2.8:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.2.8:8080" job="ecom-api-nodeport" namespace="ecom" pod="ecom-api-5748dc499c-xpzz7" service="ecom-api-nodeport"	11.978s ago	3.340ms	
http://10.244.2.10:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.2.10:8080" job="ecom-api" namespace="ecom" pod="ecom-api-5748dc499c-rsm8b" service="ecom-api"	3.838s ago	6.813ms	
http://10.244.1.8:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.1.8:8080" job="ecom-api-nodeport" namespace="ecom" pod="ecom-api-5748dc499c-6mlr2" service="ecom-api-nodeport"	2.267s ago	143.317ms	
http://10.244.2.10:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.2.10:8080" job="ecom-api-nodeport" namespace="ecom" pod="ecom-api-5748dc499c-rsm8b" service="ecom-api-nodeport"	1.181s ago	108.682ms	
http://10.244.1.6:8080/metrics	UP	container="ecom-api" endpoint="http" instance="10.244.1.6:8080" job="ecom-api"	5.898s ago	4.468ms	



Contribution: Adildabek (lead) — Prometheus rules, dashboard, Alertmanager routing; Artem added the in-app metric instrumentation.

6. Step 4 — SRE Operations (15 pt)

SLI / SLO definition

Documented in full in [docs/slo.md](#) . Summary:

SLI	SLO	Error budget (28d)
Availability (non-5xx ratio)	99.5%	≈ 3h 21m
Latency p95 (/api/products*)	< 300 ms	5% of 5-min windows
Latency p95 (/api/orders)	< 500 ms	5% of 5-min windows
Order success rate	≥ 99%	1% of orders

We also published an **error-budget policy** ([docs/slo.md](#) §"Error-budget policy") with four states from "ship freely" to "code freeze + page on-call".

Auto-scaling

k8s/04-hpa.yaml — HorizontalPodAutoscaler v2:

- `minReplicas: 2`, `maxReplicas: 8`
- CPU target: 60% utilization
- Memory target: 75% utilization
- `behavior.scaleUp.policies` — add up to 2 pods per 30s; 30s stabilization
- `behavior.scaleDown.policies` — remove 1 pod per 60s; **180s stabilization** to avoid flapping after the load drops

`metrics-server` is installed by Terraform with `--kubelet-insecure-tls` so it works on Kind without manual cert wiring.

Load testing

`loadtest/locustfile.py` defines:

1. A realistic traffic mix (70% list products, 20% product detail, 10% place order, plus a CPU-burner endpoint to demonstrate scaling).
2. A **staged spike** `LoadTestShape` : - 0–60s: ramp to 30 users - 60–180s: steady at 80 users - 180–300s: spike to 250 users (triggers HPA) - 300–420s: cool down to 80 users - 420–480s: drain to 10 users

While the spike runs we observe: - Request rate climbs in Grafana - HPA scales `ecom-api` from 2 → 6 pods within ~90 seconds - p95 stays under the SLO target except for a short transient at the start of the spike (before HPA reacts) — this is exactly the kind of evidence the error budget exists for

PowerShell - kubectl get hpa,pods (during Locust load)

PS> kubectl -n ecom get hpa,pods -o wide

NAME			REFERENCE			TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/ecom-api			Deployment/ecom-api			cpu: 51%/60%, memory: 11%/75%	2	8	8	7m11s
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod/ecom-api-5748dcd99c-6mlr2	1/1	Running	0	69s	10.244.1.8	sre-final-worker2	<none>		<none>	
pod/ecom-api-5748dcd99c-gc98s	1/1	Running	0	2m10s	10.244.1.6	sre-final-worker2	<none>		<none>	
pod/ecom-api-5748dcd99c-kw2ss	1/1	Running	0	7m11s	10.244.2.7	sre-final-worker	<none>		<none>	
pod/ecom-api-5748dcd99c-rmzb7	1/1	Running	0	7m11s	10.244.1.5	sre-final-worker2	<none>		<none>	
pod/ecom-api-5748dcd99c-rsm8b	1/1	Running	0	69s	10.244.2.10	sre-final-worker	<none>		<none>	
pod/ecom-api-5748dcd99c-tx6h5	1/1	Running	0	100s	10.244.1.7	sre-final-worker2	<none>		<none>	
pod/ecom-api-5748dcd99c-vgjcl	1/1	Running	0	100s	10.244.2.9	sre-final-worker	<none>		<none>	
pod/ecom-api-5748dcd99c-xpcz7	1/1	Running	0	2m10s	10.244.2.8	sre-final-worker	<none>		<none>	

Locust/stats/requests JSON (live)

```
PS> curl.exe -s http://localhost:8089/stats/requests
```

```
{
  "current_response_time_percentiles": {
    "response_time_percentile_0.5": 42,
    "response_time_percentile_0.95": 180
  },
  "errors": [
    {
      "error": "RemoteDisconnected('Remote end closed connection without response')",
      "first_seen": 1779018852.5715263,
      "last_seen": 1779018923.881759,
      "method": "GET",
      "name": "GET /api/work",
      "occurrences": 3
    }
  ],
  "fail_ratio": 0.0006025306286402892,
  "state": "running",
  "stats": [
    {
      "avg_content_length": 297.0,
      "avg_response_time": 62.65368566999758,
      "current_fail_per_sec": 0.0,
      "current_rps": 59.4,
      "max_response_time": 2249.0,
      "median_response_time": 14,
      "method": "GET",
      "min_response_time": 3.0,
      "name": "GET /api/products",
      "num_failures": 0,
      "num_requests": 2910,
      "response_time_percentile_0.95": 100,
      "response_time_percentile_0.99": 2100,
      "total_fail_per_sec": 0.0,
      "total_rps": 33.30570654011379
    },
    {
      "avg_content_length": 58.93414634146342,
      "avg_response_time": 65.19544487814639,
      "current_fail_per_sec": 0.0,
      "current_rps": 17.8,
      "max_response_time": 2332.0,
      "median_response_time": 14,
      "method": "GET",
      "min_response_time": 3.0,
      "name": "GET /api/products/{id}",
      "num_failures": 0,
      "num_requests": 820,
      "response_time_percentile_0.95": 98,
      "response_time_percentile_0.99": 2100,
      "total_fail_per_sec": 0.0,
      "total_rps": 9.385113183124847
    },
    {
      "avg_content_length": 32.69203747072599,
      "avg_response_time": 179.8242941453584,
      "current_fail_per_sec": 0.1,
      "current_rps": 18.4,
      "max_response_time": 2428.0,
      "median_response_time": 130,
      "method": "GET",
      "min_response_time": 30.0,
      "name": "GET /api/work",
      "num_failures": 3,
      "num_requests": 854,
      "response_time_percentile_0.95": 270,
      "response_time_percentile_0.99": 2200,
      "total_fail_per_sec": 0.03433577993826164,
      "total_rps": 9.774252022425145
    },
    {
      "avg_content_length": 31.0,
      "avg_response_time": 64.80625443067696,
      "current_fail_per_sec": 0.0,
      "current_rps": 7.6,
      "max_response_time": 2236.0,
      "median_response_time": 15,
      "method": "POST",
      "min_response_time": 4.0,
      "name": "POST /api/orders",
      "num_failures": 0,
      "num_requests": 395,
      "response_time_percentile_0.95": 94,
      "response_time_percentile_0.99": 2100,
      "total_fail_per_sec": 0.0,
      "total_rps": 4.5208776918711155
    },
    {
      "avg_content_length": 191.35569391444065,
      "avg_response_time": 83.34021044386955,
      "current_fail_per_sec": 0.1,
      "current_rps": 103.2,
      "max_response_time": 2428.0,
      "median_response_time": 20,
      "method": "",
      "min_response_time": 3.0,
      "name": "Aggregated",
      "num_failures": 3,
      "num_requests": 4979,
      "response_time_percentile_0.95": 180,
      "response_time_percentile_0.99": 2100,
      "total_fail_per_sec": 0.0006025306286402892,
      "total_rps": 33.30570654011379
    }
  ]
}
```

```
    "total_fail_per_sec": 0.03433577993826164,  
    "total_rps": 56.9859494375349  
  },  
  ],  
  "total_fail_per_sec": 0.03433577993826164,  
  "total_rps": 56.9859494375349,  
  "user_count": 80  
}
```

Contribution: Artem (lead) — HPA tuning + Locust profile; Adildabek validated SLO derivation and dashboard correlation during the spike.

7. Step 5 — Documentation & Repository Hygiene (10 pt)

- `README.md` — quick start, repo layout, rubric mapping
- `docs/architecture.md` — system diagram + design rationale
- `docs/slo.md` — SLI/SLO definitions + error-budget policy
- `docs/runbook.md` — playbook for every alert in the system
- `terraform/README.md` — how to reproduce / how to swap state backend
- `loadtest/README.md` — how to drive load and what to look for
- `.gitignore` — Terraform state, Go binaries, Python venv, IDE files
- Per-folder READMEs where it adds value

The repository is structured by concern (`app/` , `terraform/` , `k8s/` , `monitoring/` , `loadtest/` , `docs/` , `scripts/`) — easy to navigate during the defense.

8. Defense & Demo — Plan (30 pt)

We will run the live demo in this order (~8 minutes):

Time	Action	What the audience sees
0:00	<code>./scripts/bootstrap.sh</code> (or show the pre-built cluster)	Terraform output, pods Ready
1:00	<code>./scripts/smoke-test.sh</code>	API responses, metrics endpoint
1:30	Open Grafana, walk the SRE Golden Signals dashboard	Live SLI panels
2:30	Open Prometheus → Targets, then Alerts	<code>ecom-api</code> UP, alert rules loaded
3:00	Open Alertmanager UI	Empty (good) — explain receivers
3:30	Start <code>locust -f locustfile.py</code> with the staged spike	Traffic ramping in Grafana
5:00	<code>kubectl -n ecom get hpa,pods -w</code> next to Grafana	HPA scales from 2 → 6 pods
6:30	Show Alertmanager when <code>EcomApiHighCpu</code> (or latency) fires	Live alert

Time	Action	What the audience sees
7:30	Open the GitHub Actions run + <code>ghcr.io</code> package	CI/CD evidence

Per-member talking points

- **Adildabek** — IaC walkthrough (`terraform/` file by file), Prometheus Operator pattern, dashboard derivation from SLIs, Alertmanager routing.
- **Artem** — Go service instrumentation, Dockerfile (distroless, non-root), CI/CD secrets model, HPA tuning, Locust shape design.

Each member should be ready to answer one deep-dive question from the other's area; preparation notes are in our shared doc.

9. Known limitations & next steps

Item	Why it's not in scope	What we'd do next
Persistent storage (DB)	The brief only requires demonstrating SRE practices on the <i>service</i>	Add Postgres via the Bitnami chart + a PVC, then refactor <code>store</code> to use it
Multi-AZ / multi-region	Local Kind cluster	Migrate Terraform to EKS module with two AZ subnets, replace <code>helm_release</code> calls 1:1
Real Slack/PagerDuty integration	Demoing webhook receivers locally	Replace the placeholder URLs in <code>AlertmanagerConfig</code> with org-issued Slack webhook
End-to-end TLS	Out of grading rubric scope	Add <code>cert-manager</code> + an Ingress controller
SLO burn-rate alerts (multi-window multi-burn)	Beyond the depth covered in lectures	Add Google SRE workbook recipes once we have ≥ 28 days of metrics

10. Appendix — Reproducibility checklist

- ☐ Docker Desktop running
- ☐ `terraform`, `kubectl`, `kind`, `python` installed
- ☐ `./scripts/bootstrap.sh` exits 0

- ☐ All pods in `ecom` and `monitoring` namespaces are `Ready`
 - ☐ `http://localhost:30030` loads Grafana, dashboard "ecom-api — SRE Golden Signals" is present
 - ☐ `http://localhost:30090/targets` shows `ecom-api` UP
 - ☐ `http://localhost:30080` loads Alertmanager
 - ☐ `http://localhost:30081/api/products` returns the seed product list
 - ☐ `locust ...` drives traffic and HPA scales above `minReplicas`
 - ☐ GitHub Actions latest run is green
 - ☐ `ghcr.io/RendersC/ecom-api:<sha>` is present in the packages tab
-

End of report.